

Année universitaire 2025–2026

RAPPORT DE PROJET

MLP, CNN et RNN : Trois Approches de Deep Learning pour la Classification et la Modélisation de Séquences

Réalisé par : Nisrine Amroug

Encadré par : Mme. HIDILA Zineb

Filière : 4IAD

Table des matières

Table des matières	2
1. Introduction	3
2. Partie 1 — MLP : Classification binaire sur le dataset Netflix Shows.....	4
2.1 Contexte et objectifs.....	4
2.2 Environnement et outils	4
2.3 Prétraitement des données	4
2.4 Architecture du modèle	4
2.5 Entraînement	5
2.6 Évaluation et démonstration interactive	5
3. Partie 2 — MLP vs CNN : Classification d'images sur Fashion-MNIST	7
3.1 Contexte et objectifs.....	7
3.2 Méthodologie générale.....	7
3.3 Modèle baseline : MLP.....	7
3.4 Modèle CNN (inspiré de LeNet-5).....	7
3.5 Expériences architecturales	8
3.6 Visualisation des cartes de caractéristiques	8
3.7 Comparaison finale et résultats	9
4. Partie 3 — RNN : Modèle de langage pour la traduction Anglais–Français.....	11
4.1 Contexte et objectifs.....	11
4.2 Configuration initiale (Setup).....	11
4.3 Préparation des données	11
4.4 Architecture du modèle : RNN simple	12
4.5 Entraînement et évaluation	12
4.6 Perspectives : LSTM, GRU et traduction séquence-à-séquence	13
5. Discussion transversale.....	14
6. Conclusion générale.....	15
7. Références bibliographiques.....	16

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du module Deep Learning. Le projet explore trois familles d'architectures de réseaux de neurones — le Perceptron Multicouche (MLP), les Réseaux de Neurones Convolutifs (CNN) et les Réseaux de Neurones Récurents (RNN) — à travers trois cas d'usage distincts, permettant de couvrir aussi bien des données tabulaires que des images et des séquences textuelles.

L'objectif global est double : d'une part, mettre en pratique les concepts fondamentaux du deep learning (prétraitement, conception d'architecture, entraînement, évaluation) sur des problèmes concrets ; d'autre part, comparer empiriquement le comportement et les performances de ces architectures afin de mettre en évidence l'influence du biais inductif de chacune sur la tâche à laquelle elle est appliquée.

Le rapport est organisé en trois parties expérimentales, suivies d'une discussion transversale et d'une conclusion :

- Partie 1 — Un MLP appliqué à une tâche de classification binaire sur le dataset Netflix Shows (données tabulaires).
- Partie 2 — Une comparaison entre un MLP et un CNN (inspiré de LeNet-5) pour la classification d'images sur le dataset Fashion-MNIST.
- Partie 3 — Un modèle de langage RNN entraîné sur un corpus de paires de phrases Anglais–Français (fra.txt), première étape vers un système de traduction séquence-à-séquence.

Chaque partie suit la même trame méthodologique : présentation du contexte et des objectifs, préparation des données, conception de l'architecture, entraînement du modèle, puis évaluation et analyse des résultats.

2. Partie 1 — MLP : Classification binaire sur le dataset Netflix Shows

2.1 Contexte et objectifs

Cette première partie a pour objectif d'appliquer un Perceptron Multicouche (MLP) à un problème de classification binaire sur des données tabulaires réelles, issues du catalogue Netflix. La tâche consiste à prédire une caractéristique cible (par exemple le type de contenu) à partir des métadonnées disponibles pour chaque titre, en mettant l'accent sur l'ensemble du pipeline : depuis le nettoyage des données brutes jusqu'à la démonstration interactive du modèle entraîné.

2.2 Environnement et outils

Le tableau suivant résume les ressources, bibliothèques et outils utilisés pour cette partie du projet.

Catégorie	Outil / Ressource	Rôle
Dataset	Netflix Shows (Kaggle, shivamb/netflix-shows)	Données tabulaires sur le catalogue Netflix
Accès aux données	kagglehub	Téléchargement programmatique du dataset
Manipulation de données	pandas	Nettoyage, transformation et analyse des données
Prétraitement	scikit-learn (LabelEncoder, StandardScaler, train_test_split)	Encodage, normalisation, séparation train/test
Deep Learning	PyTorch (torch)	Construction, entraînement et évaluation du MLP
Visualisation	matplotlib, seaborn	Analyse exploratoire des données (EDA)
Environnement	Google Colaboratory	Développement et exécution du notebook

2.3 Prétraitement des données

Le prétraitement constitue une étape déterminante pour les données tabulaires. Les opérations suivantes ont été appliquées :

- Gestion des valeurs manquantes dans les colonnes du jeu de données.
- Feature engineering : extraction de composantes à partir des dates (année, mois) et parsing de la durée des contenus (conversion en valeur numérique exploitable).
- Encodage des variables catégorielles via Label Encoding et One-Hot Encoding selon la nature de chaque variable.
- Mise à l'échelle des variables numériques avec StandardScaler, appliquée après séparation des ensembles afin d'éviter toute fuite de données (data leakage).

2.4 Architecture du modèle

Le modèle retenu est un Perceptron Multicouche implémenté avec PyTorch. Les couches denses successives permettent de combiner les features tabulaires prétraitées pour produire une prédiction binaire en sortie.

```

-----
Layer (type)          Output Shape          Param #
-----
Linear-1              [-1, 64]              48,320
Linear-2              [-1, 1]               65
-----
Total params: 48,385
Trainable params: 48,385
Non-trainable params: 0
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.18
Estimated Total Size (MB): 0.19
-----

```

Figure 1: model summary

2.5 Entraînement

L'entraînement repose sur la fonction de perte Binary Cross-Entropy with Logits (BCEWithLogitsLoss), adaptée à la classification binaire car elle intègre directement la sigmoïde de manière numériquement stable. L'optimiseur Adam a été utilisé pour la mise à jour des poids, avec un cycle d'entraînement (forward pass, calcul de la perte, rétropropagation, mise à jour) répété sur plusieurs époques.

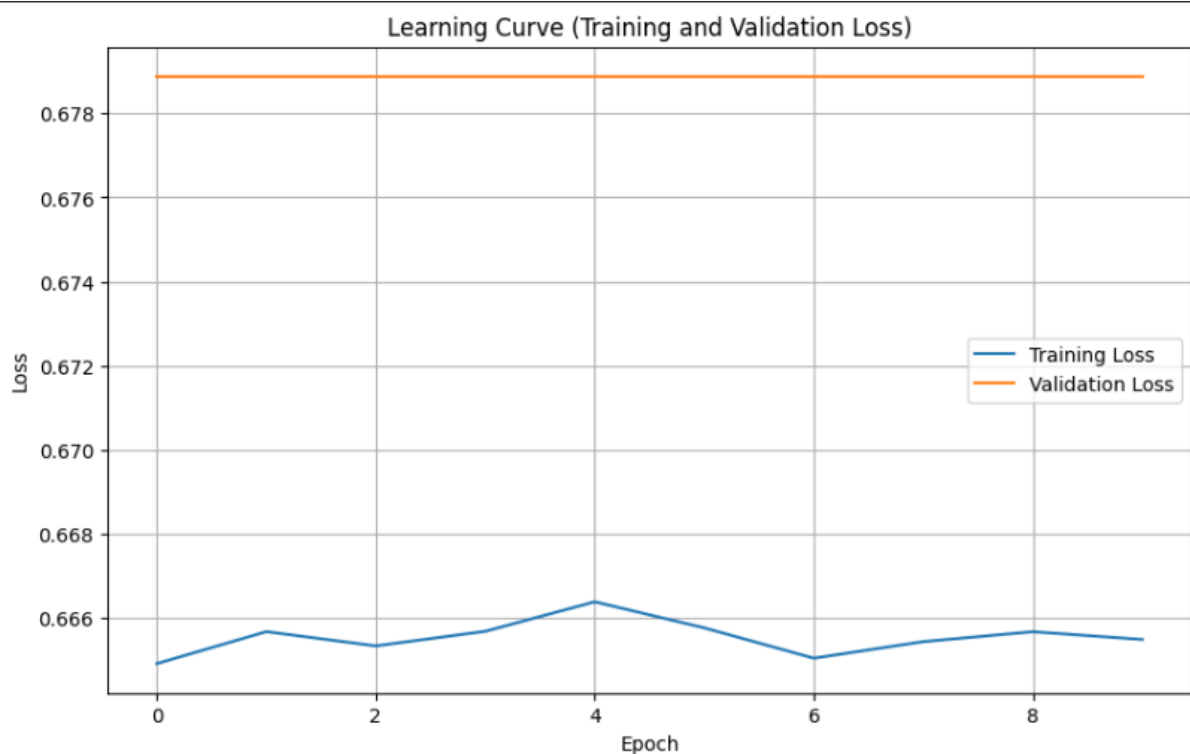


Figure 2: la courbe d'apprentissage

2.6 Évaluation et démonstration interactive

Le modèle a été évalué à l'aide de la métrique d'accuracy sur l'ensemble de test. Une fonctionnalité de prédiction interactive a également été mise en place, permettant de saisir manuellement les caractéristiques d'un nouveau contenu et d'obtenir en temps réel la prédiction du modèle entraîné, illustrant ainsi concrètement l'inférence du MLP.

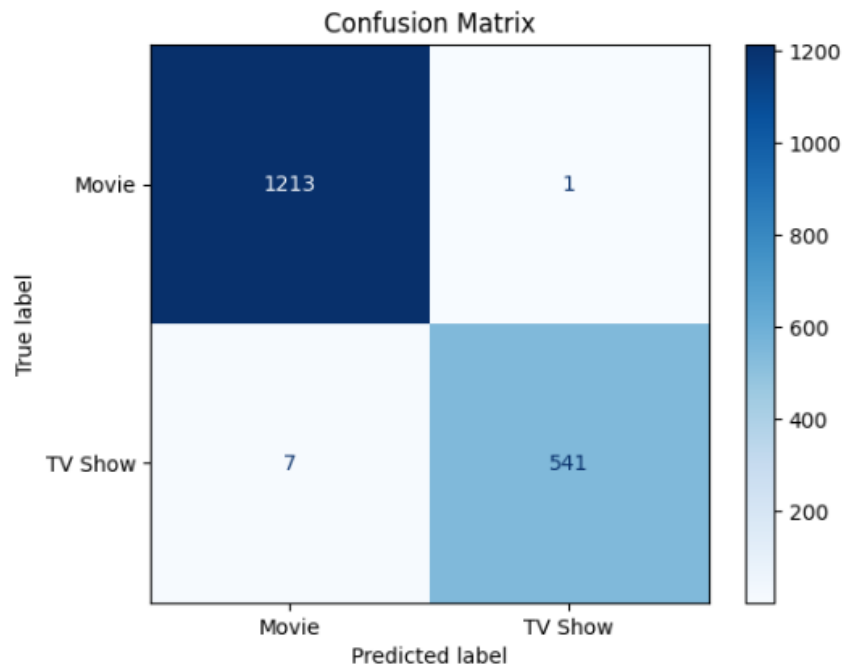


Figure 3:matrice de confusion

À compléter : valeurs exactes d'accuracy obtenues sur le jeu de test, et brève analyse des erreurs de classification observées.]

3. Partie 2 — MLP vs CNN : Classification d'images sur Fashion-MNIST

3.1 Contexte et objectifs

Cette deuxième partie explore et compare les performances des réseaux de neurones multicouches (MLP) et des réseaux de neurones convolutifs (CNN) pour une tâche de classification d'images, en utilisant le jeu de données Fashion-MNIST. L'objectif principal est de démontrer l'efficacité supérieure des CNN dans le traitement des données visuelles et d'analyser l'impact de différents choix architecturaux sur leurs performances.

3.2 Méthodologie générale

Le projet a été structuré en plusieurs phases clés, résumées ci-dessous.

1. Initialisation et préparation : configuration de l'environnement (Google Colab), importation des bibliothèques (torch, torchvision, numpy, matplotlib, seaborn, pandas, sklearn) et fixation des graines aléatoires pour la reproductibilité des expériences.
2. Implémentations manuelles : implémentation "from scratch" des opérations de convolution 2D (corr2d) et de pooling 2D (pool2d), afin d'approfondir la compréhension des mécanismes fondamentaux des CNN.
3. Chargement et exploration des données : chargement, normalisation et séparation de Fashion-MNIST en ensembles d'entraînement, de validation et de test, avec visualisation d'échantillons.
4. Définition des utilitaires d'entraînement : développement d'une fonction générique `train_model`, intégrant la validation et un mécanisme d'arrêt anticipé (early stopping).
5. Modèle baseline MLP : conception, entraînement et évaluation d'un MLP simple traitant les images comme des vecteurs plats, ignorant leur structure spatiale.
6. Modèle CNN (LeNet) : implémentation, entraînement et évaluation d'une architecture inspirée de LeNet-5, exploitant convolutions et pooling pour capturer des caractéristiques hiérarchiques.
7. Expériences architecturales : test de variantes de l'architecture CNN (padding, stride, type de pooling, nombre de filtres, convolutions 1×1).
8. Visualisation des cartes de caractéristiques : visualisation des activations internes du CNN pour comprendre comment il perçoit les images.
9. Comparaison finale et analyse synthétique : comparaison détaillée MLP/CNN (matrices de confusion, précision, F1-score, nombre de paramètres) et rédaction d'une analyse de la supériorité des CNN.

3.3 Modèle baseline : MLP

Le MLP traite chaque image comme un vecteur de pixels aplati, sans tenir compte de la structure spatiale bidimensionnelle de l'image. Ce modèle sert de référence (baseline) pour évaluer l'apport des couches convolutives du CNN.

3.4 Modèle CNN (inspiré de LeNet-5)

L'architecture CNN repose sur une succession de blocs convolutifs (convolution → activation → pooling), permettant d'extraire progressivement des caractéristiques de plus en plus abstraites tout en réduisant la dimension spatiale. Contrairement au MLP, le CNN exploite le partage de poids et la localité des filtres, ce qui lui permet de détecter des motifs visuels indépendamment de leur position dans l'image.

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 6, 28, 28]	156
BatchNorm2d-2	[-1, 6, 28, 28]	12
ReLU-3	[-1, 6, 28, 28]	0
AvgPool2d-4	[-1, 6, 14, 14]	0
Conv2d-5	[-1, 16, 10, 10]	2,416
BatchNorm2d-6	[-1, 16, 10, 10]	32
ReLU-7	[-1, 16, 10, 10]	0
AvgPool2d-8	[-1, 16, 5, 5]	0
Flatten-9	[-1, 400]	0
Linear-10	[-1, 120]	48,120
ReLU-11	[-1, 120]	0
Dropout-12	[-1, 120]	0
Linear-13	[-1, 84]	10,164
ReLU-14	[-1, 84]	0
Linear-15	[-1, 10]	850
Total params: 61,750		
Trainable params: 61,750		
Non-trainable params: 0		
Input size (MB): 0.00		
Forward/backward pass size (MB): 0.16		
Params size (MB): 0.24		
Estimated Total Size (MB): 0.40		

Figure 4: résumé du modèle

3.5 Expériences architecturales

Plusieurs variantes du CNN ont été testées pour évaluer l'impact des choix architecturaux :

- Padding et stride : influence sur la taille des cartes de caractéristiques en sortie de chaque couche convolutive.
- Type de pooling : comparaison entre Max Pooling et Average Pooling.
- Nombre de filtres : impact de l'augmentation du nombre de filtres par couche sur la capacité du modèle.
- Convolutions 1×1 : utilisation pour la réduction de dimension des canaux.

L'analyse de ces expériences a mis en évidence l'importance de certains choix de conception, notamment le Max Pooling, qui offre une meilleure invariance à la translation et préserve les activations les plus fortes.

3.6 Visualisation des cartes de caractéristiques

Pour mieux comprendre le fonctionnement interne du CNN, les cartes de caractéristiques (feature maps) produites par les différentes couches convolutives ont été visualisées, permettant d'observer la transition de filtres détectant des contours simples vers des motifs plus abstraits dans les couches profondes.

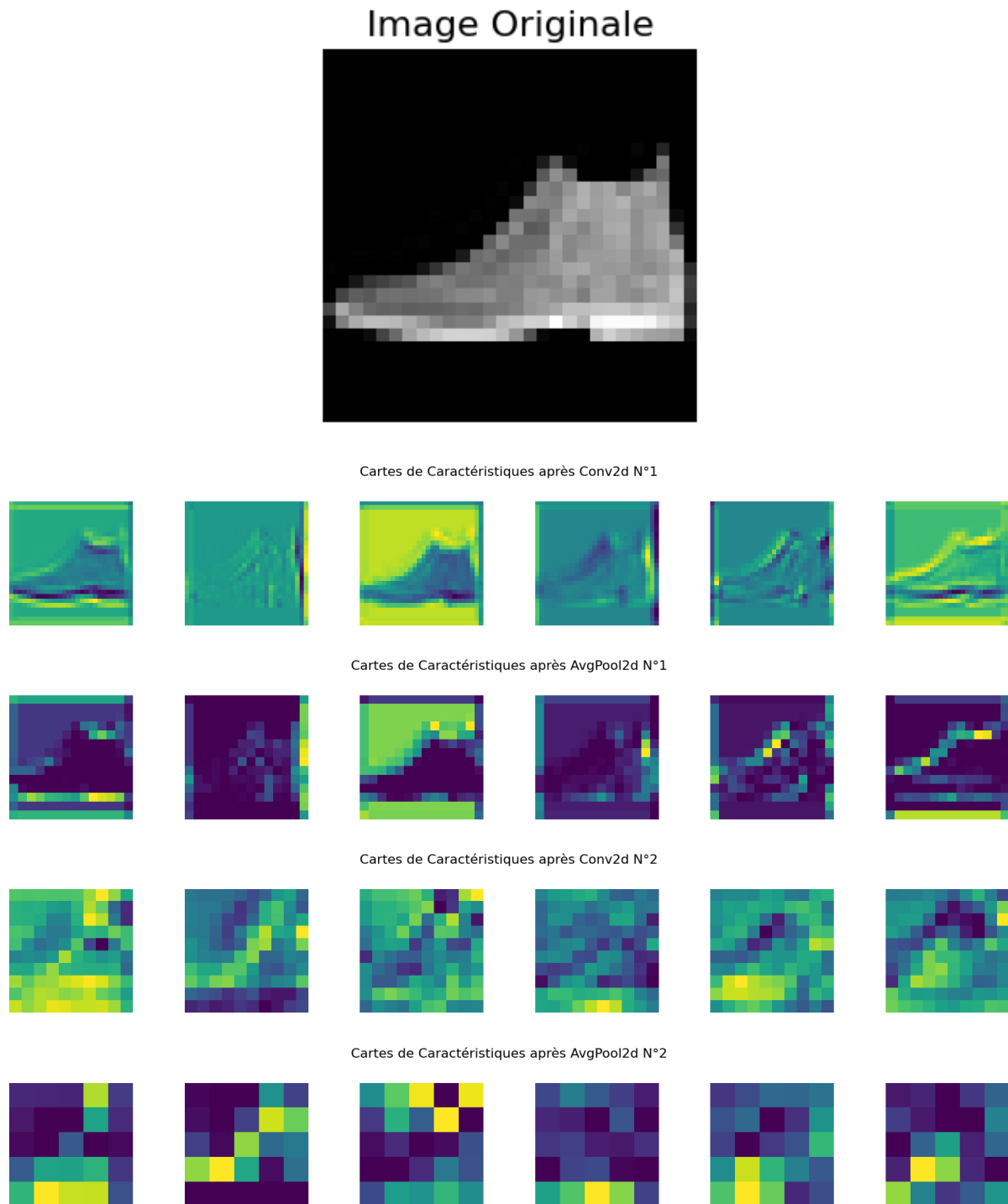


Figure 5: visualisations des cartes de caractéristiques extraites des différentes couches du CNN

3.7 Comparaison finale et résultats

La comparaison finale entre le MLP et le CNN s'appuie sur plusieurs critères : la précision (accuracy), le F1-score, les matrices de confusion ainsi que le nombre total de paramètres de chaque modèle.

Critère	MLP (baseline)	CNN (LeNet)
Accuracy (test)	[à compléter]	[à compléter]
F1-score	[à compléter]	[à compléter]
Nombre de paramètres	[à compléter]	[à compléter]
Temps d'entraînement	[à compléter]	[à compléter]

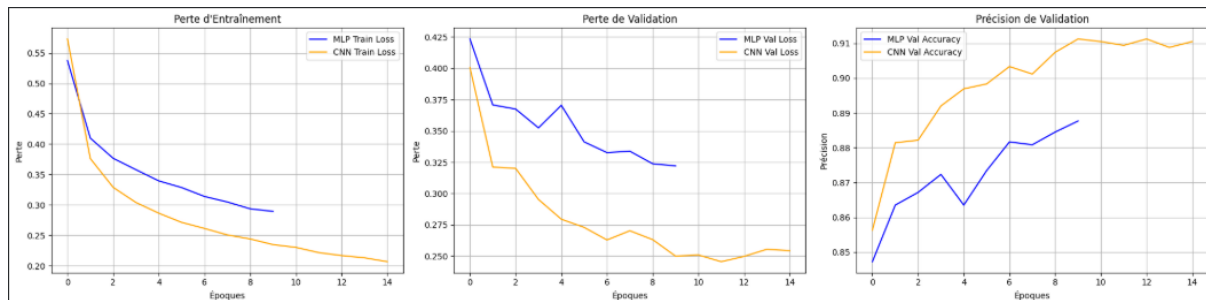


Figure 6 : les matrices de confusion du MLP et du CNN, les courbes d'apprentissage comparées.

Nos expériences ont démontré la supériorité des CNN pour la classification d'images : le CNN atteint une précision plus élevée tout en utilisant un nombre de paramètres significativement réduit par rapport au MLP. Cette efficacité s'explique par la capacité des CNN à exploiter la localité spatiale et le partage des poids, permettant aux filtres de détecter des caractéristiques indépendamment de leur position dans l'image.

Bien que performants, les CNN présentent certaines limites, notamment des difficultés à distinguer des classes visuellement proches (par exemple certains types de hauts vestimentaires dans Fashion-MNIST) ainsi qu'une sensibilité connue aux attaques adverses. Ces limites n'enlèvent toutefois rien au constat général : pour la classification d'images standard, l'efficacité des CNN surpasse largement celle des MLP.

4. Partie 3 — RNN : Modèle de langage pour la traduction Anglais–Français

4.1 Contexte et objectifs

Cette troisième partie aborde le traitement de données séquentielles à l'aide de réseaux de neurones récurrents (RNN), dans la perspective de la traduction automatique. Le corpus utilisé (fra.txt) contient des paires de phrases Anglais–Français. À ce stade du projet, l'accent a été mis sur la mise en place du pipeline de données et l'entraînement d'un premier modèle de langage RNN simple, ouvrant la voie à des architectures plus avancées (LSTM, GRU, encodeur-décodeur) dans une étape ultérieure.

4.2 Configuration initiale (Setup)

L'environnement de travail a été initialisé avec les bibliothèques nécessaires : torch pour le deep learning, nltk pour le traitement du langage naturel, pandas pour la manipulation des données, ainsi que matplotlib et seaborn pour la visualisation. Le périphérique de calcul (GPU ou CPU) a été détecté automatiquement, et les graines aléatoires ont été fixées à 42 afin d'assurer la reproductibilité des résultats.

Le fichier fra.txt, contenant les paires de phrases Anglais-Français, a été importé puis chargé dans un DataFrame pandas. Le dataset a été restreint aux 15 000 premières paires afin de limiter le temps d'entraînement. Une fonction de nettoyage de texte a ensuite été appliquée aux phrases anglaises et françaises : conversion en minuscules et suppression de toute ponctuation à l'exception des apostrophes, afin d'homogénéiser les données avant l'étape de vectorisation.

4.3 Préparation des données

La transformation du texte nettoyé en représentation numérique exploitable par le modèle a nécessité plusieurs étapes :

- Définition de tokens spéciaux : <pad>, <bos>, <eos> et <unk>, avec leurs indices associés.
- Construction de deux objets Vocabulary distincts (anglais et français), recensant les mots uniques du corpus. Les mots apparaissant moins de deux fois (min_freq = 2) ont été remplacés par le token <unk>.
- Encodage des phrases : conversion des mots en identifiants numériques, ajout des tokens <bos> et <eos>, puis padding ou troncature de chaque séquence à une longueur fixe MAX_LEN = 20.
- Séparation du dataset en ensembles d'entraînement (80%), de validation (10%) et de test (10%).
- Création d'objets TranslationDataset et de DataLoader personnalisés, avec une taille de batch BATCH_SIZE = 32, pour une itération efficace sur les données durant l'entraînement.

Des histogrammes ont également été générés pour visualiser la distribution des longueurs de phrases en anglais et en français, apportant un éclairage utile sur les caractéristiques du corpus.

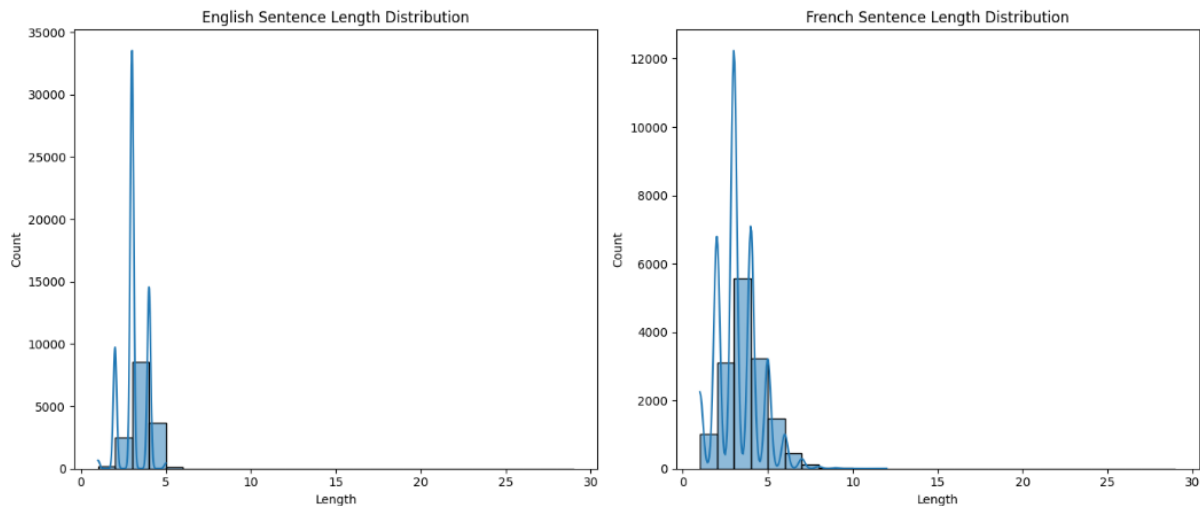


Figure 7: histogrammes des longueurs de phrases (anglais et français).

4.4 Architecture du modèle : RNN simple

Un modèle de langage RNN simple a été implémenté et entraîné pour la tâche de prédiction du mot suivant (next-token prediction) sur la portion anglaise du dataset. La classe `RNNDecoder` est composée d'une couche d'embedding, d'une couche récurrente `nn.RNN` à une seule couche (`hidden_dim = 128`), et d'une couche linéaire de sortie projetant l'état caché vers la distribution de probabilité sur le vocabulaire.

```
...
Simple RNN Model Summary:
RNNDecoder(
  (embedding): Embedding(1864, 256, padding_idx=0)
  (rnn): RNN(256, 128, batch_first=True)
  (fc_out): Linear(in_features=128, out_features=1864, bias=True)
  (dropout): Dropout(p=0.0, inplace=False)
)
The simple RNN model has 767,048 trainable parameters

Training Simple RNN Language Model...
Epoch: 01 | Train Loss: 3.870 | Train PPL: 47.93 | Val Loss: 3.357 | Val PPL: 28.71
Epoch: 02 | Train Loss: 3.152 | Train PPL: 23.38 | Val Loss: 3.129 | Val PPL: 22.86
Epoch: 03 | Train Loss: 2.926 | Train PPL: 18.66 | Val Loss: 3.013 | Val PPL: 20.35
Epoch: 04 | Train Loss: 2.790 | Train PPL: 16.28 | Val Loss: 2.942 | Val PPL: 18.95
Epoch: 05 | Train Loss: 2.694 | Train PPL: 14.79 | Val Loss: 2.899 | Val PPL: 18.15
Epoch: 06 | Train Loss: 2.622 | Train PPL: 13.76 | Val Loss: 2.868 | Val PPL: 17.60
Epoch: 07 | Train Loss: 2.569 | Train PPL: 13.05 | Val Loss: 2.840 | Val PPL: 17.11
Epoch: 08 | Train Loss: 2.525 | Train PPL: 12.49 | Val Loss: 2.834 | Val PPL: 17.01
Epoch: 09 | Train Loss: 2.491 | Train PPL: 12.07 | Val Loss: 2.808 | Val PPL: 16.58
Epoch: 10 | Train Loss: 2.461 | Train PPL: 11.71 | Val Loss: 2.804 | Val PPL: 16.51

Simple RNN Language Model Training Complete.
```

Figure 8: résumé du modèle

4.5 Entraînement et évaluation

Les fonctions `train_language_model` et `evaluate_language_model` ont été implémentées pour calculer la perte, appliquer l'écrouissage de gradient (gradient clipping, `max_norm = 1.0`) et calculer la perplexité, métrique standard pour l'évaluation des modèles de langage. Le modèle a été entraîné sur 10 époques sur la portion anglaise du dataset.

Les courbes de perplexité d'entraînement et de validation ont été tracées afin de suivre la progression de l'apprentissage et la capacité de généralisation du modèle. Les normes des gradients (avant et après écrêtage) ont également été visualisées pour les trois premières époques, ce qui est essentiel pour évaluer la stabilité de l'entraînement et l'effet du gradient clipping.

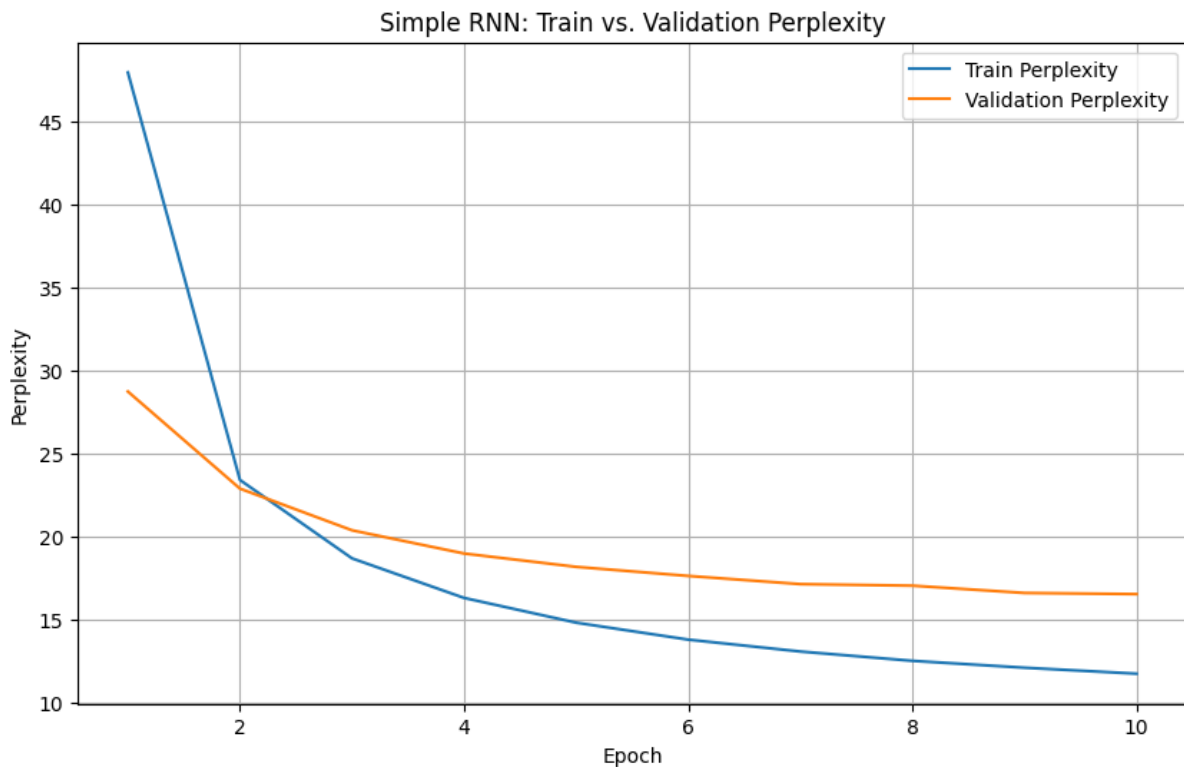


Figure 9: les courbes de perplexité (entraînement/validation)

4.6 Perspectives : LSTM, GRU et traduction séquence-à-séquence

Le RNN simple constitue une première brique de référence, mais il est connu pour souffrir du problème de disparition du gradient sur les séquences longues. Les prochaines étapes du projet consisteront à implémenter et comparer des architectures LSTM et GRU sur la même tâche de modélisation de langage, avant d'assembler un véritable système encodeur-décodeur pour la traduction complète de l'anglais vers le français.

5. Discussion transversale

Les trois parties de ce projet permettent de mettre en perspective les forces et les limites respectives du MLP, du CNN et du RNN, chacun étant adapté à un type de données et porteur d'un biais inductif différent.

Architecture	Données cibles	Biais inductif	Cas d'usage dans ce projet
MLP	Tabulaires	Aucun	Classification binaire (Netflix Shows)
CNN	Images, grilles 2D	Spatial (localité)	Classification d'images (Fashion-MNIST)
RNN	Séquences	Temporel (ordre)	Modèle de langage (corpus Anglais–Français)

Le MLP, dépourvu de biais inductif particulier, s'avère adapté aux données tabulaires du dataset Netflix Shows où il n'existe pas de structure spatiale ou temporelle à exploiter. En revanche, appliqué à des images dans la Partie 2, il se montre nettement moins performant que le CNN, car il ignore la structure spatiale des pixels et nécessite un nombre de paramètres bien plus important pour atteindre des performances comparables.

Le CNN, grâce à son biais inductif spatial (localité des filtres et partage des poids), exploite efficacement la structure en grille des images et obtient de meilleurs résultats avec moins de paramètres, comme démontré dans la comparaison MLP/CNN sur Fashion-MNIST.

Le RNN, quant à lui, est conçu pour exploiter l'ordre séquentiel des données, ce qui en fait un choix naturel pour la modélisation du langage. Toutefois, sa version simple atteint rapidement ses limites sur des séquences longues à cause de la disparition du gradient, justifiant le recours futur à des variantes comme le LSTM ou le GRU.

Cette mise en perspective confirme un principe central du deep learning appliqué : le choix de l'architecture doit être guidé par la structure intrinsèque des données et non par une préférence arbitraire — un MLP performant sur des données tabulaires n'est pas nécessairement adapté à des images, et inversement.

6. Conclusion générale

Ce projet a permis de mettre en œuvre et de comparer trois architectures fondamentales du deep learning à travers trois cas d'usage représentatifs : la classification binaire sur données tabulaires (MLP), la classification d'images (MLP vs CNN) et la modélisation de séquences textuelles (RNN).

Au-delà de l'implémentation, ce travail a permis d'approfondir la compréhension des mécanismes internes de chaque architecture — notamment grâce à l'implémentation manuelle des opérations de convolution et de pooling — ainsi que des bonnes pratiques associées à l'entraînement de modèles de deep learning : prétraitement rigoureux, choix de fonction de perte adaptée, gestion du gradient, suivi des courbes d'apprentissage et évaluation multi-métrique.

Les résultats obtenus confirment empiriquement l'importance du biais inductif dans le choix d'une architecture : la supériorité du CNN sur le MLP pour les données visuelles, et la pertinence du RNN pour les données séquentielles. Les travaux futurs porteront sur l'extension du modèle de langage RNN vers des architectures LSTM et GRU, puis vers un système complet de traduction automatique séquence-à-séquence Anglais–Français.

7. Références bibliographiques

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP 2014*.

Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.

Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15, 1929–1958.

Ioffe, S. & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML 2015*.

PyTorch Documentation officielle : <https://pytorch.org/docs/stable/>

Scikit-learn : <https://scikit-learn.org/stable/>

Kaggle Dataset — Netflix Shows : [shivamb/netflix-shows](https://www.kaggle.com/shivamb/netflix-shows).